

Contents

Projet Final — Prédiction du Churn Telco	1
Mission 0 — Cadrage du problème	1
Mission 1 — Données et analyse exploratoire	2
Mission 2 — Préparation et pipeline sans fuite de données	3
Mission 3 — Modélisation et comparaison rigoureuse	4
Mission 4 — Optimisation, calibration et interprétabilité	5
Mission 5 — Déploiement, tests et monitoring	6
Réponses aux questions de réflexion	8

Projet Final — Prédiction du Churn Telco

Auteur : [Votre nom] · **Cours :** Supervised Learning, Master IA · **Enseignant :** Ibrahima SY

Mission 0 — Cadrage du problème

Problème métier

Prédire, pour chaque client actif, sa probabilité de résiliation dans le mois suivant, afin que le service marketing priorise les offres de rétention vers les clients à risque, sous contrainte de budget de campagne limité (impossibilité de contacter toute la base).

Définition de la cible

Churn (documentation IBM) = client ayant résilié au cours du dernier mois. Il s'agit donc d'un churn constaté a posteriori ; le modèle sera appliqué en production sur des clients dont l'issue est encore inconnue. **Point de vigilance pour la Mission 1 :** toute variable connue uniquement après la résiliation (motif de départ, date de clôture...) constituerait une fuite de données directe.

Coûts d'erreur (estimation chiffrée)

Statistiques réelles du dataset : ARPU moyen (MonthlyCharges) = 64,76 €/mois. Hypothèse de marge nette sectorielle : ~30 % du CA (benchmark télécom).

Erreur	Hypothèse	Calcul	Coût estimé
Faux Négatif (churner manqué)	Perte de marge sur ~9 mois avant récupération durable par un concurrent	$9 \times (0,30 \times 64,76\text{€})$	$\approx 175 \text{ €}$
Faux Positif (offre à un fidèle)	Remise -20 % pendant 3 mois + coût de contact	$(0,20 \times 64,76\text{€} \times 3) + 8\text{€}$	$\approx 47 \text{ €}$

Ratio FN/FP $\approx 3,7$ → le rappel doit être privilégié sur la précision, sans l'ignorer.

Hypothèses de travail transparentes (pas des données réelles d'entreprise), posées pour fixer un ratio de coût défendable qui pilote les choix méthodologiques ultérieurs.

Métrique

- **Principale (comparaison de modèles, M2/M3) : F2-score** — pondère le rappel $\sim 4\times$ plus que la précision, cohérent avec le ratio de coût ci-dessus. L'accuracy est écartée : un modèle "toujours non-churn" obtient 73 % d'accuracy pour zéro valeur métier.

- **Secondaire n°1 : PR-AUC** — indépendante du seuil, plus informative que ROC-AUC sous déséquilibre modéré (~27 % de churn).
- **Secondaire n°2 : coût métier attendu** = $FN \times 175\text{€} + FP \times 47\text{€}$ (pour N clients scorés) — arbitre final du choix de seuil en Mission 4 ; le F2-score n'est qu'un proxy d'entraînement.
- **Seuil de réussite fixé a priori** : $F2 \geq 0,65$ en validation croisée, $\text{Recall} \geq 75\%$. Objectif de départ, à confronter à la baseline de la Mission 2.

Nuance opérationnelle : le budget de campagne étant limité, le déploiement réel s'apparente à un classement (top-K clients par score) plutôt qu'à un seuil de probabilité universel — justifie l'intérêt de la PR-AUC / lift chart comme diagnostics dès maintenant.

Risques & hypothèses

- **Stationarité** : marché télécom concurrentiel (guerre des prix MVNO/5G) → distribution probablement instable au-delà de 6-12 mois. Justifie le plan de monitoring (Mission 5).
- **RGPD** : le score alimente une action commerciale avec un agent humain dans la boucle de décision finale → hors du champ strict de l'Art. 22 RGPD (décision automatisée sans intervention humaine). Contrainte de design : outil d'aide à la décision, pas déclencheur automatique.
- **Latence** : usage principal en scoring batch périodique (campagnes hebdo/mensuelles) ; pas de SLA temps réel dur. L'API /predict sert à l'intégration CRM et aux tests.
- **Explicabilité** : exigence d'adoption et de non-discrimination (des humains agissent différemment selon le score) — justifie SHAP comme exigence, pas comme bonus (Mission 4).

Mission 1 — Données et analyse exploratoire

Notebook complet : notebooks/01_edu.ipynb (exécuté, sorties visibles).

Profiling

7 043 lignes, 21 colonnes. 0 doublon (lignes complètes et customerID). Aucune feature quasi-constante (seuil 98%).

Le piège TotalCharges : typée object, elle contient 11 valeurs vides correspondant **exactement** aux 11 clients avec `tenure == 0` (nouveaux clients non encore facturés). Ce n'est pas un manquant aléatoire mais un zéro logique — décision retenue pour la Mission 2 : imputation par 0 (connaissance métier), pas par la médiane.

Colinéarité : TotalCharges est quasi-déterministe ($r = 0,9996$) à partir de `tenure * MonthlyCharges`. Piste Mission 2 : remplacer par un ratio `TotalCharges / (tenure+1)` plutôt que de garder les 3 variables brutes corrélées.

Détection de fuite de données

Méthodologie : AUC univarié par feature, seuil suspect fixé à 0,90. Résultat : le maximum observé est 0,74 (Contract, tenure) — **aucune fuite détectée**. Toutes les colonnes sont des attributs de compte connus pendant que le client est actif ; aucune ne décrit un événement postérieur à la résiliation.

Analyse bivariée — top 5 features (information mutuelle)

`Contract > tenure > OnlineSecurity > TechSupport > InternetService`. `gender`, `PhoneService`, `MultipleLines` : pouvoir discriminant quasi nul ($MI \approx 0$, $AUC \approx 0,50$) — candidats à l'élimination en M2, à confirmer par CV.

Hypothèses formulées et vérifiées (5/5 confirmées)

Hypothèse	Résultat	Statut
Contrat mensuel → plus de churn	42,7% vs 11,3% vs 2,8% (mensuel/1an/2ans)	Confirmée
Faible ancienneté → plus de churn	médiane 10 mois (churn) vs 38 mois (fidèles)	Confirmée
Pas d'OnlineSecurity → plus de churn	41,8% vs 14,6%	Confirmée
Fibre optique → plus de churn	41,9% vs 19,0% (DSL) vs 7,4% (aucun)	Confirmée
Chèque électronique → plus de churn	45,3% vs 15-19% (autres modes)	Confirmée (effet le plus fort)

3 insights majeurs

1. Contract est le signal dominant (MI la plus élevée, AUC univarié 0,74).
2. TotalCharges est redondante et son "manquant" est un zéro logique, pas un NA statistique.
3. Aucune fuite de données — la vigilance en M2 portera sur l'ordre des opérations (split avant transformation), pas sur une colonne à retirer.

Mission 2 — Préparation et pipeline sans fuite de données

Notebook complet : notebooks/02_pipeline_baseline.ipynb. Pipeline réutilisable : src/pipeline.py + src/features.py.

Split d'abord

train_test_split stratifié (80/20, random_state=42) réalisé sur les données encore brutes. Taux de churn identique à 4 décimales entre train (0,2654) et test (0,2654). Si les statistiques d'imputation/encodage étaient apprises avant ce split, le test "verrait" indirectement des informations du train — le score de test ne mesurerait plus une vraie généralisation.

Pipeline

ColumnTransformer : sous-pipeline numérique (imputation médiane + StandardScaler) et catégoriel (imputation mode + OneHotEncoder), assemblés dans un Pipeline scikit-learn unique. Les statistiques (médiane, mode, modalités) ne sont apprises qu'au .fit(X_train), jamais sur le test — garantie structurelle, pas seulement disciplinaire.

Le correctif TotalCharges=0 (Mission 1) est appliqué via un transformer dédié (TotalChargesFixer) **avant** le ColumnTransformer : il n'apprend aucune statistique du train (règle fixe), donc aucun risque de fuite même s'il précède le split.

Baseline

F2 = 0,5621 (± 0,0364) en CV 5-fold stratifiée sur le train — régression logistique. Comparaison imputation médiane vs métier : écart marginal (+0,0002, attendu vu que seules 11/5634 lignes du train sont concernées) — la décision reste justifiée sur le plan conceptuel, pas seulement statistique.

Feature engineering — résultat honnête : aucun gain retenu

Feature testée	F2 CV	Gain vs baseline
Baseline (aucune FE)	0,5621	—
+ tenure_group	0,5579	−0,0042
+ n_services	0,5617	−0,0005
+ charges_per_tenure (remplace TotalCharges)	0,5506	−0,0116
Toutes combinées	0,5469	−0,0153

Les trois features candidates **dégradent** légèrement le score sur ce modèle linéaire : le OneHotEncoder + StandardScaler capture déjà l'essentiel de l'information qu'elles tentaient de resynthétiser. **Rasoir d'Occam appliqué** : aucune n'est retenue dans le pipeline final. Décision réévaluée en Mission 3 avec des modèles non linéaires (arbres), susceptibles d'exploiter des seuils/interactions différemment.

F2 de référence pour la Mission 3 : 0,5621 ($\pm$ 0,0364).

Mission 3 — Modélisation et comparaison rigoureuse

Notebook complet : notebooks/03_modeling_comparison.ipynb.

Baseline naïve

DummyClassifier (classe majoritaire) : **F2 = 0,0000** (rappel nul, accuracy ~73,5% mais valeur métier nulle). Barre minimale triviale à battre — confirme que l'accuracy seule serait un piège d'évaluation ici.

Trois modèles, trois familles (mêmes 5 folds stratifiés)

Modèle	Justification	F2 (CV 5-fold)	Écart-type
Régression logistique (linéaire)	Référence interprétable, peu de risque d'overfitting	0,5621	0,0364
Forêt aléatoire (arbres)	Capture nativement interactions/seuils non linéaires	0,5084	0,0210
SVM RBF (marge maximale)	3 ^e famille distincte, tractable à cette taille de dataset (~5600 lignes)	0,5265	0,0140

Modèle le plus stable : SVM (écart-type le plus faible), mais **meilleure moyenne : régression logistique** — compromis performance/stabilité explicite, pas de vainqueur trivial.

Pourquoi la régression logistique gagne : vérifié via PR-AUC (indépendante du seuil) — elle est aussi la meilleure (0,658 vs 0,608 RF vs 0,633 SVM), donc ce n'est pas un artefact du seuil à 0,5 mais un vrai avantage de pouvoir de classement. Interprétation : le signal dominant (Contract, tenure, OnlineSecurity) est largement monotone et peu bruité (cf. Mission 1) — terrain favorable au modèle linéaire. RF (profondeur non limitée) et SVM (hyperparamètres par défaut) pourraient progresser après tuning (M4).

Test statistique (Wilcoxon apparié)

À 5 plis : $p = 0,0625$ pour LogReg vs les deux autres modèles — **non significatif à 5%**, mais c'est une limite de puissance du test (5 paires $\rightarrow$ p minimal atteignable = 0,0625, même en cas de victoire sur tous les plis). À **10 plis** : LogReg bat SVM sur 9/10 plis, **$p = 0,0039$** vs SVM et **$p = 0,0098$** vs Forêt aléatoire — significatif dans les deux cas. **Modèle retenu : régression logistique.**

Retest du feature engineering (promesse M2) avec Forêt aléatoire

Gains marginaux (+0,0016 tenure_group, +0,0017 charges_per_tenure), très inférieurs à l'écart-type inter-plis ($\sim 0,02$) — **non significatifs même avec un modèle non linéaire**. Confirme la décision de la Mission 2 : le signal est déjà capté par les variables brutes, indépendamment du modèle.

Analyse d'erreurs

Profil des faux négatifs (churners manqués, $n=682/5634$) : ancienneté médiane **25 mois**, contre **6 mois** pour les churners correctement détectés. Le modèle excelle sur le pattern « nouveau client + contrat mensuel » mais **rate systématiquement les clients fidèles de longue date qui finissent par partir** — profil différent, mal capturé par des variables statiques (instantané, pas de série temporelle). Limite du dataset, pas du modèle : des features de tendance (évolution récente de charges, interactions support) seraient nécessaires pour ce segment — hors du périmètre disponible ici.

Mission 4 — Optimisation, calibration et interprétabilité

Notebook complet : notebooks/04_tuning_calibration_shap.ipynb.

Tuning Optuna — 2 modèles retunés, ≥ 6 hyperparamètres, pruner médian

Conformément à la promesse de la Mission 3, **les deux finalistes ont été retunés** (pas seulement le vainqueur) : régression logistique (7 hyperparamètres : C, penalty, l1_ratio, class_weight, tol, max_iter, fit_intercept, solveur saga fixé) et forêt aléatoire (7 hyperparamètres : n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features, class_weight, criterion). Recherche interne 3-fold avec MedianPruner (39/50 essais complétés pour LogReg, 30/70 pour RF — le pruner a effectivement écarté des combinaisons non prometteuses).

Modèle	F2 défaut (M3)	F2 tuné	Gain
Régression logistique	0,5621	0,7221	+0,1599
Forêt aléatoire	0,5084	0,7233	+0,2149

Importance des hyperparamètres (fANOVA) : class_weight domine dans les deux cas (76% LogReg, 53% RF) — le gain n'est pas un raffinement fin de régularisation, mais la correction du déséquilibre de classes que les valeurs par défaut ignoraient.

Résultat clé : après tuning, l'écart entre les deux modèles disparaît (Wilcoxon LogReg vs RF tuné : $p = 0,625$, non significatif) — le tuning a neutralisé l'avantage structurel de la régression logistique observé en Mission 3 à hyperparamètres par défaut. **Modèle retenu : régression logistique tunée** — à performance statistiquement équivalente, le modèle le plus simple et interprétable est préféré (rasoir d'Occam), cohérent avec l'exigence d'explicabilité posée dès la Mission 0.

Calibration

`class_weight='balanced'` déforme les probabilités prédites (le modèle agit comme si les classes étaient équilibrées). Vérifié par reliability diagram : le modèle non calibré est **surconfiant** dans les probabilités hautes (ex. prédit 0,88 pour une fréquence réelle observée de 0,76). **Brier score : 0,1653 → 0,1353 après calibration Platt (sigmoid)**, soit -18,1%. Platt préféré à l'isotonique (plus robuste à cette taille de dataset, ~5 600 échantillons). Le modèle final déployé utilisera les probabilités calibrées.

Interprétabilité SHAP

Calculé sur le modèle tuné non calibré (LinearExplainer) — la calibration Platt étant une transformation monotone, elle ne change ni le classement ni l'attribution des features, seulement l'échelle affichée.

Top features (importance globale) : tenure domine largement, suivi de MonthlyCharges, InternetService_Fiber optic, TotalCharges, Contract_Month-to-month. Cohérent avec les Missions 1 et 3. InternetService_Fiber optic a un effet positif net sur le risque indépendamment de MonthlyCharges — pas seulement un effet prix.

3 décisions individuelles : le cas le plus instructif est le Faux Positif — un profil **quasiment identique** au Vrai Positif (nouveau client, contrat mensuel, fibre chère, proba 0,92) mais qui est resté fidèle. Rappel important : le churn reste probabiliste, pas déterministe — le modèle n'a pas "mal raisonné", il a identifié un profil à risque objectivement élevé qui, cette fois, ne s'est pas concrétisé. Ce n'est pas un bug corrigible par plus de features, mais une limite structurelle de la prédiction.

Dependence plot tenure : relation monotone décroissante, pente plus marquée pour les faibles anciennetés — le risque varie le plus vite chez les nouveaux clients. La coloration par MonthlyCharges révèle une interaction : à ancienneté égale, payer plus cher accentue légèrement l'impact SHAP — cohérent avec l'hypothèse « nouveau client + facture élevée = risque maximal » de la Mission 1.

Seuil de décision

Optimisé sur le coût métier de la Mission 0 (FN=175€, FP=47€), balayé sur les probabilités calibrées, en CV sur le train (test toujours non touché).

Seuil	Precision	Recall	% base ciblée	Coût / 1000 clients
0,50 (défaut)	0,653	0,537	21,8%	25 057 €
0,18 (optimal)	0,457	0,892	51,8%	18 240 €

Le seuil optimal fait passer le rappel de 54% à 89% au prix d'une précision plus faible — cohérent avec le ratio de coût FN/FP $\approx 3,7$ établi en Mission 0.

Tension avec la contrainte opérationnelle (déjà anticipée en M0) : à ce seuil, 52% de la base est ciblée. Si le budget de campagne ne permet pas de contacter la moitié des clients, le seuil de coût pur n'est pas directement actionnable. Recommandation opérationnelle : utiliser le score calibré pour **classer** les clients et contacter le top-K compatible avec le budget réel — le seuil 0,18 sert de référence théorique, pas de règle rigide de production.

Mission 5 — Déploiement, tests et monitoring

Code : `src/model.py`, `src/train.py`, `tests/test_pipeline.py`, `api/main.py`.

Évaluation finale sur le test — premier et unique contact

Le test (20%, tenu à l'écart depuis la Mission 2) est utilisé pour la première fois ici. **F2 = 0,7475** (vs 0,7221 en CV train) — aucun signe de sur-apprentissage, le test confirme même légèrement mieux que la CV. Une fois cette évaluation actée, le modèle de production est réentraîné sur **100% des données disponibles** (pratique standard : maximiser la performance déployée sans invalider l'évaluation déjà obtenue sur données non vues).

Sérialisation

`joblib.dump` du pipeline complet (preprocessing + calibration + modèle). Vérifié : prédictions strictement identiques après rechargement (`np.array_equal`).

Tests (pytest) — 8 tests, tous passants

Sortie de bonne forme, probabilités dans [0,1], gestion des valeurs manquantes, schéma de features attendu (`EXPECTED_FEATURES`), erreur explicite si feature manquante, performance sur jeu de référence $\geq$ seuil d'alerte (0,60), cohérence après rechargement, et un test de sanity métier (profil archétypal à risque > profil fidèle, avec bornes absolues 0,5/0,2).

API REST (FastAPI) — testée en conditions réelles (curl)

GET `/health`, POST `/predict` (validation Pydantic stricte — un Contract invalide renvoie **422** avec message explicite, pas un plantage), GET `/model-info`. Exemples curl complets dans `README.md`.

Documentation

`README.md` (installation, reproduction, usage API) + `MODEL_CARD.md` (structure Mitchell et al., FAccT 2019). **Qu'est-ce qu'une model card et pourquoi un standard ?** Une model card documente, de façon structurée et systématique, le contexte d'usage prévu, les données d'entraînement, les métriques **par sous-groupe** (pas seulement globales) et les limites connues d'un modèle — elle rend explicite ce qu'un rapport de performance global masque. Elle est devenue un standard car elle force à documenter les angles morts *avant* le déploiement plutôt que de les découvrir en production, et donne aux équipes non techniques (juridique, produit) un document actionnable.

Découverte concrète en construisant la model card : le taux de ciblage diffère nettement entre seniors (80,2%) et non-seniors (47,9%) — écart réel, cohérent avec un taux de churn presque doublé chez les seniors, mais qui mérite une revue conformité avant un déploiement à grande échelle (détail complet dans `MODEL_CARD.md`).

Plan de monitoring

- **Data drift** (distribution des features en entrée) : surveiller la distribution de `Contract`, `InternetService`, `MonthlyCharges` — un changement de mix d'offres commerciales fausserait silencieusement les prédictions. Outil : **Evidently** (rapports de drift automatisés, tests statistiques par colonne).
- **Concept drift** (relation features → cible qui change) : le modèle a été entraîné sur un marché à un instant T (cf. Mission 0 — non-stationarité déjà identifiée comme risque). Suivre la performance réelle sur les clients dont l'issue devient connue (churné ou non 1 mois après scoring) par comparaison à la performance de référence.
- **Performance drift** : suivre F2/recall/precision glissants sur les clients scorés puis observés, alerte si dégradation > 10% relatif vs la référence de la Mission 4.
- **Fréquence** : rapport Evidently hebdomadaire (aligné sur le cycle des campagnes marketing identifié en Mission 0), ré-entraînement complet si drift confirmé sur 2 semaines consécutives.

Réponses aux questions de réflexion

1. Concept drift. Un modèle bon à l'entraînement se dégrade en production quand la relation entre features et cible change alors que le modèle reste figé — par exemple si un concurrent lance une offre agressive qui change ce qui pousse un client à partir. C'est différent du data drift (la distribution des features change, pas leur lien avec la cible). À surveiller : performance réelle glissante sur clients dont l'issue est connue (pas seulement la distribution des features en entrée, qui peut rester stable alors que la relation sous-jacente a changé).

2. Information mutuelle vs corrélation de Pearson. Pearson ne mesure que les relations **linéaires** et n'est définie que pour des variables numériques. L'information mutuelle capture **toute** forme de dépendance statistique (linéaire ou non, monotone ou non) et s'applique nativement aux variables catégorielles — c'est ce qui a permis, en Mission 1, de classer Contract (catégorielle) comme la feature la plus prédictive, un résultat qu'un simple tableau de corrélations de Pearson n'aurait même pas pu produire.

3. No Free Lunch. Aucun algorithme n'est universellement meilleur car la performance d'un modèle dépend de la structure du problème (linéarité du signal, taille du dataset, bruit) — un modèle adapté à une structure donnée sera sous-optimal sur une autre. **Preuve empirique obtenue dans ce projet** : à hyperparamètres par défaut (Mission 3), la régression logistique bat nettement la forêt aléatoire et le SVM, parce que le signal du churn Telco est largement monotone. Après tuning (Mission 4), l'écart disparaît complètement (Wilcoxon $p=0,625$) — ni le modèle linéaire ni les modèles non linéaires n'ont de supériorité intrinsèque ; leur position relative dépend entièrement du réglage et du problème. C'est exactement pourquoi un benchmark comparatif honnête (mêmes folds, test statistique) est indispensable plutôt que de choisir un modèle par réputation.

4. Fairness. Deux métriques : la **demographic parity** (le taux de ciblage — prédictions positives — doit être comparable entre sous-groupes, indépendamment du taux de churn réel) et l'**equalized odds** (le rappel et le taux de faux positifs doivent être comparables entre sous-groupes, cette fois en tenant compte du taux réel). Mesuré dans ce projet (MODEL_CARD.md) : quasi-parité de genre (52,8% vs 53,3% de ciblage), mais écart net par âge (SeniorCitizen : 80,2% vs 47,9%) — expliqué par un taux de churn réel presque double chez les seniors (equalized odds plutôt respecté), mais qui viole la demographic parity stricte. Les deux métriques peuvent être en tension : les satisfaire simultanément n'est généralement pas possible si les taux de base diffèrent entre groupes — un arbitrage explicite, documenté, est nécessaire plutôt qu'un optimum supposé automatique.

5. Fuite de données — où elle aurait pu s'introduire, et comment elle a été évitée. - *Imputation/encodage calculés avant le split* → évité par un Pipeline scikit-learn unique, `fit()` uniquement sur le train (Mission 2). - *TotalCharges imputé par une statistique globale (médiane calculée sur tout le dataset)* → remplacé par une règle métier fixe (0 si `tenure==0`), qui n'apprend rien du train et peut donc être appliquée sans risque même avant le split (Mission 1-2). - *Feature engineering apprenant une statistique (ex. un target encoding)* → aucune des features créées n'en utilisait ; toutes étaient déterministes (comptage, ratio, bucketing à seuils fixes), donc sans risque même si calculées hors pipeline — choix documenté explicitement en Mission 2. - *Sélection de modèle/hyperparamètres sur le test* → jamais : tout le tuning (Mission 4) et la comparaison de modèles (Mission 3) se sont faits en validation croisée sur le train ; le test n'a été touché qu'une seule fois, en Mission 5, pour l'évaluation finale. - *Vérification active, pas seulement procédurale* : Mission 1 a quantifié l'absence de fuite via un AUC univarié par feature (seuil 0,90), pas juste une relecture visuelle des colonnes.